

Streaming Framework

(Part 1)

OUTLINE

- ① Introduction to the streaming model
- ② Warm-up: finding the majority element
- ③ Sampling
 - Uniform sampling with reservoir sampling
 - Finding (approximate) frequent items
- ④ Sketching (Part 2)
 - Counting distinct elements with probabilistic counting
 - Estimating individual frequencies and the second moment
- ⑤ Filtering: Bloom filters for membership problem (Part 2)

Dealing with volume and velocity

Typical scenario:

- The data to be processed arrive as a **continuous stream**
 - because they are **generated by some evolving (possibly endless) process** (e.g., **tweets**, **buy/sell orders**, **sensor data**);
 - or because their **massive volume discourages random accesses** (e.g., sequential **secondary/tertiary** data access)
- Therefore: **data analysis** must happen **on the fly** using limited memory, without storing all data for subsequent offline processing.
- **Many applications** (some examples next)

Some applications

Network management:

- **Stream:** packets routed through a router.
- **Task:** gather traffic statistics (e.g., average number of connections/second to same IP address)

Online auctions:

- **Stream:** online auctions generate data on objects on sale, bids, ...
- **Tasks:** statistical estimation of auction data (e.g., average selling price over the items sold by each seller, highest bid in a given period of time, ...)

Some applications

Internet of Things:

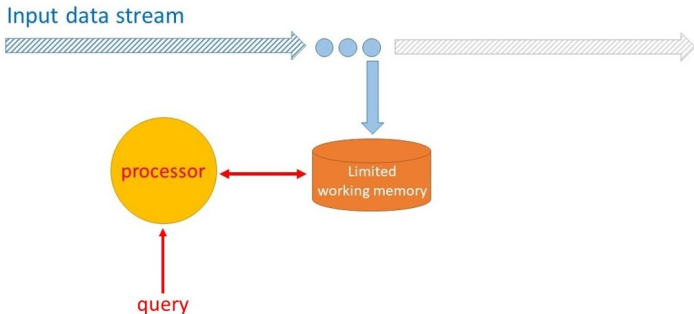
- **Stream:** data generated by thousands/millions of sensors.
- **Tasks:** learn from data, outlier detection, gather statistics,...

Sequential disk accesses:

- **Stream:** data from a massive file on secondary storage (sequential access rates are higher than random access),
- **Tasks:** data processing of data in the file. In this case it is also possible to sequentially scan the same file twice or even more times (**multi-pass**).

Streaming Model

- (Sequential) machine with **limited amount of working memory**.
- Input provided as a **continuous (one-way) stream**.



Streaming Model

Let $\Sigma = x_1, x_2, \dots, x_n \dots$ denote the **input stream** received sequentially, where x_i is the i -th element received. Upon receiving x_n :

- Suitable data structures stored in the working memory are updated (**UPDATE task**)
- If required, a solution for the problem at hand, relative to the input set $x_1, x_2, \dots, x_n$ is computed from the data stored in the working memory (**QUERY task**)

The following **Key Performance Indicators** are usually considered

- **KPI 1:** Size s of the working memory (**aim:** $s \ll |\Sigma|$)
- **KPI 2:** Number p of sequential passes over Σ (**aim:** $p = 1$)
- **KPI 3:** Update time T_u per item (**aim:** $T_u = O(1)$)
- **KPI 4:** Query time T_q to return a solution to instance $\{x_1, \dots, x_n\}$ (**aim:** T_q independent of n)

Streaming Model

Algorithm Design Techniques

- **Approximate solutions** (exact ones may require excessive space)
- Maintain **lossy summary** of Σ via a **synopsis** data structure (e.g., **random sample**, **hash-based sketch**)

Typical data analysis tasks

- **Identification of frequent items** (or frequent patterns).
- **Useful statistics**: e.g., frequency moments, quantiles, histograms
- **Optimization and graph problems**: e.g., clustering, triangle counting

Goal: suitable tradeoffs between accuracy, working memory, update/query time (i.e., *throughput*).

Warm-up:
finding the majority element in a stream

Warm-up: finding the majority element

Majority problem

Given a stream $\Sigma = x_1, x_2, \dots, x_n$, return the element x (if any) that occurs $> n/2$ times in Σ .

Examples:

- In $\Sigma = A, B, A, C, A, D, A, A$, the majority element is A .
- In $\Sigma = A, B, C, D, D, E, E$, a majority element does not exist.

Standard off-line setting. Easily solved using linear space in $O(n \log n)$ time, through sorting, or $O(n)$ expected time, through hashing.

Streaming setting. Need 1 or 2 passes, depending on goals

- First pass (Boyer-Moore algorithm): find an element x which is the majority element, if one exists.
- Second pass: check whether x is indeed the majority element.

Boyer-Moore algorithm

The Boyer-Moore algorithm maintains 2 integers *cand* and *count*:

- *cand* contains a candidate majority element
(*the true majority element, if one exists*);
- *count* is a counter;

Initialization: *cand* $\leftarrow$ null and *count* $\leftarrow$ 0.

For each x_t **in** Σ **do**

if *count* = 0 **then** { *cand* $\leftarrow$ x_t ; *count* $\leftarrow$ 1; }

else {

if *cand* = x_t **then** *count* $\leftarrow$ *count* + 1

else *count* $\leftarrow$ *count* - 1

 }

At the end: **return** *cand*

} Query

} Update

Example: $\Sigma = A, A, A, C, C, B, B, A, A$
 $x_1 \ x_2 \ x_3 \ x_4 \ x_5 \ x_6 \ x_7 \ x_8 \ x_9$

STEP	cand	count
0	null	0
1	A	1
2	A	2
3	A	3
4	A	2
5	A	1
6	A	0
7	B	1
8	B	0
9	A	1

At the end: return A (true majority)

Example: $\Sigma = A, A, A, C, C, C, B, B, B$

STEP	cand	count
0	null	0
1	A	1
2	A	2
3	A	3
4	A	2
5	A	1
6	A	0
7	B	1
8	B	2
9	B	3

At the end: return B, but no majority exists

Warm-up: finding the majority element

Theorem

The Boyer-Moore algorithm is *correct* (i.e., it returns the true majority, if one exists) and requires:

- *working memory of size $O(1)$;*
- *1 pass;*
- *$O(1)$ update and query times.*

} Straightforward

We need to prove only correctness

Proof. To show correctness we must consider only the case when a true majority exists. Then, let m be the true majority element in Σ , and let cand_t and count_t be the values of the 2 variables cand and count after processing $x_1 \dots x_t$. The proof is based on the following INVARIANT

For any $t \geq 0$, $x_1 x_2 \dots x_t$ can be partitioned into

- * $\text{count}_t \geq 0$ occurrences of cand_t
- * $(t - \text{count}_t)/2$ pairs (e_1, e_2) with $e_1 \neq e_2$

EXERCISE: Show by induction that the invariant holds for every $t \in [0, n]$

For instance, in the first example (Slide 12), for $t=5$ we have $x_1 \dots x_5 = A A A C C$, $\text{count}_5 = 1$, $\text{cand}_5 = A$ and, indeed, there are 1 occurrence of A and 2 pairs $(A, C)(A, C)$.

We now show that, for an arbitrary stream Σ of length n with a true majority m , at the end of the algorithm $\text{cand}_n = m$

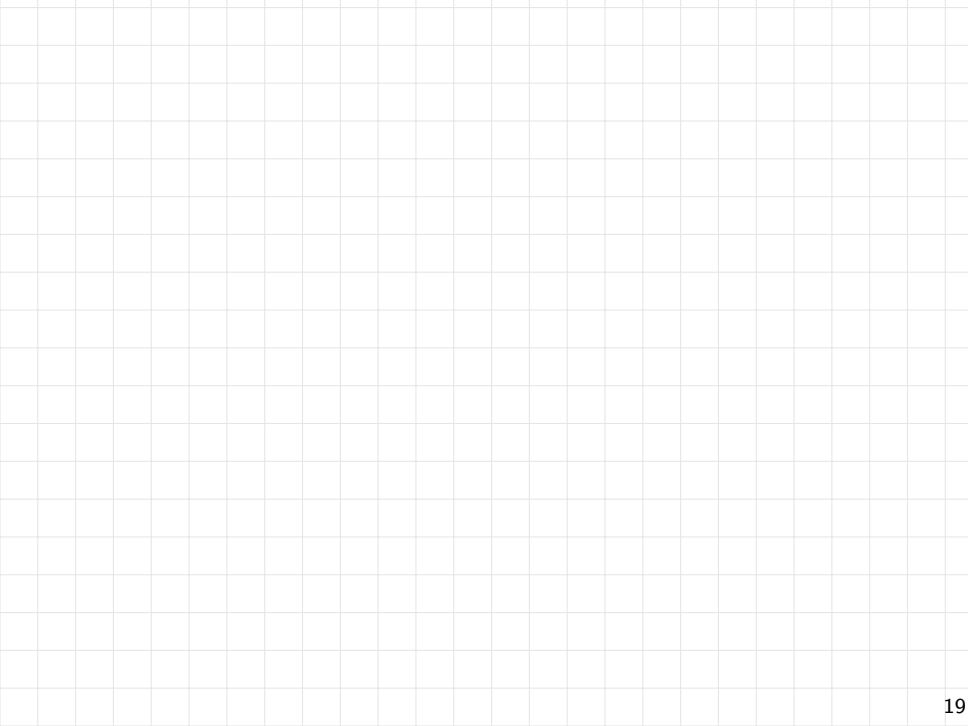
By contradiction suppose that $\text{cand}_n \neq m$
Since the invariant holds for $t=n$ (see exercise)
we have that Σ contains

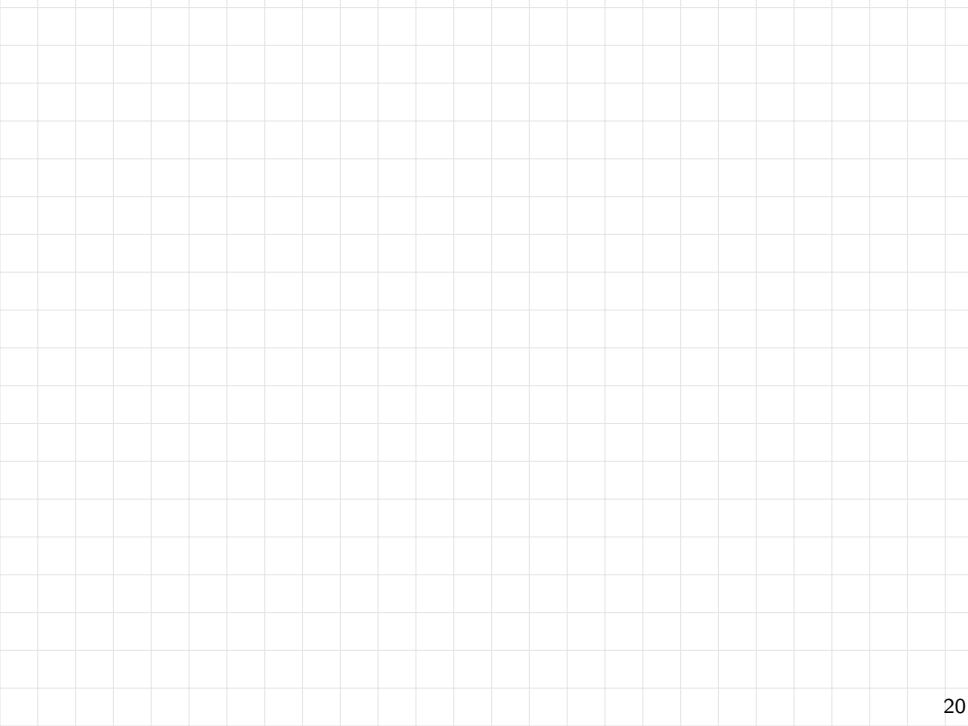
- * $\text{count}_n \geq 0$ occurrences of $\text{cand}_n \neq m$
- * $(n - \text{count}_n)/2$ pairs (e_1, e_2) with $e_1 \neq e_2$

Therefore, m can occur at most $(n - \text{count}_n)/2$
times, and since $\text{count}_n \geq 0$ we have that
$$(n - \text{count}_n)/2 \leq n/2$$

which contradicts the fact that m is the

true majority (i.e., occurs $> n/2$ times) $\square$





Exercise

Let $\Sigma = x_1, x_2, \dots, x_n$ be a stream of n distinct integers in $[1, n + 1]$. Design a streaming algorithm that finds the missing number using $O(1)$ -size working memory, 1 pass, and $O(1)$ update and query times.

Sampling

Sampling

Definition

Given a set X of n elements and an integer $1 \leq m < n$, an m -sample of X is a random subset $S \subset X$ of size m , such that for each $x \in X$, we have $\Pr(x \in S) = m/n$ (uniform sampling).

In the off-line setting, an m -sample S of X can be easily computed by selecting m elements from X (or, equivalently, their indices) with uniform probability and without replacement.

In the streaming setting, things get harder, especially in application scenarios where streams are potentially unbounded and at every time step a random sample of the elements seen so far is sought (e.g., *monitoring of sensor data*)

Sampling

Sampling problem

Given a (possibly unbounded) stream $\Sigma = x_1, x_2, \dots$ and an integer $m < |\Sigma|$, maintain, for every $t \geq m$, an m -sample S of the prefix $\Sigma_t = x_1, x_2, \dots, x_t$.

For a fixed t , the solution is easy

- Before the stream starts, compute an m -sample $\mathcal{I}$ of the set of integers $\{1, \dots, t\}$.
- For each entry $x_i \in \Sigma_t$ with $i \in \mathcal{I}$, add x_i to S .

How can we maintain S for every t ?

Reservoir Sampling: algorithm

The following algorithm, dubbed **Reservoir Sampling** was developed by J.S. Vitter in 1985.

Initialization: $S \leftarrow \emptyset$.

For each x_t **in** Σ **do**

if $t \leq m$ **then** add x_t to S ;

else with probability m/t do the following: {

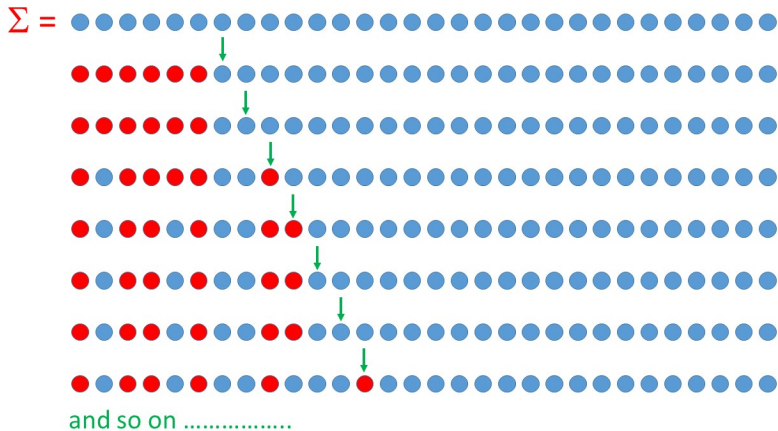
 evict an element x from S , chosen with uniform probability

 add x_t to S

 }

Remark: the stream size needs not be known, and, in fact, it can be unbounded.

Example: $m = 6$



Reservoir Sampling: analysis

Theorem

Let $\Sigma = x_1, x_2, \dots$. At any time $t \geq m$, the set S maintained by the reservoir sampling algorithm is an m -sample of $\Sigma_t = x_1, x_2, \dots, x_t$. Moreover, the algorithm features $O(1)$ update time.

Proof: the only non-trivial thing to prove is the fact that S is indeed an m -sample of $x_1 \dots x_t$

let S_t be S after processing $x_1 \dots x_t$. We show by induction on $t \geq m$ that for every $i \in [1, t]$ we have $P_i(x_i \in S_t) = m/t$

BASE for INDUCTION: $t=m$ (straightforward)

INDUCTION STEP: Suppose that the property holds at time $t-1$, that is

$$\forall i \in [1, t-1] \quad P_i(x_i \in S_{t-1}) = \frac{m}{t-1} \quad (\text{inductive Hp.})$$

We now prove that $P_2(x_i \in S_t) = \frac{m}{t} \quad \forall i \in [1, t]$

* $P_2(x_t \in S_t) = \frac{m}{t}$ by construction

* Consider an arbitrary x_i with $i < t$, and define event $A \equiv "x_t \text{ is added to } S_t \text{ at time } t"$

CLEARLY: $P_2(A) = m/t$ and $P_2(\neg A) = 1 - m/t$

By the law of total probabilities we can write

$$P_2(x_i \in S_t) = \underbrace{P_2((x_i \in S_t) \cap A)}_{(1)} + \underbrace{P_2((x_i \in S_t) \cap \neg A)}_{(2)}$$

$$(1): P_2((x_i \in S_t) \cap A) = P_2(x_i \in S_t | A) \underbrace{P_2(A)}_{= m/t}$$

$$P_2(x_i \in S_t | A) = \underbrace{P_2(x_i \in S_{t-1})}_{m/(t-1) \text{ by inductive hp.}} \times \underbrace{P_2(x_i \text{ not selected at time } t)}_{1 - 1/m}$$

$$\Rightarrow P_2((x_i \in S_t) \cap A) = \frac{m}{t-1} \cdot \left(1 - \frac{1}{m}\right) \cdot \frac{m}{t}$$

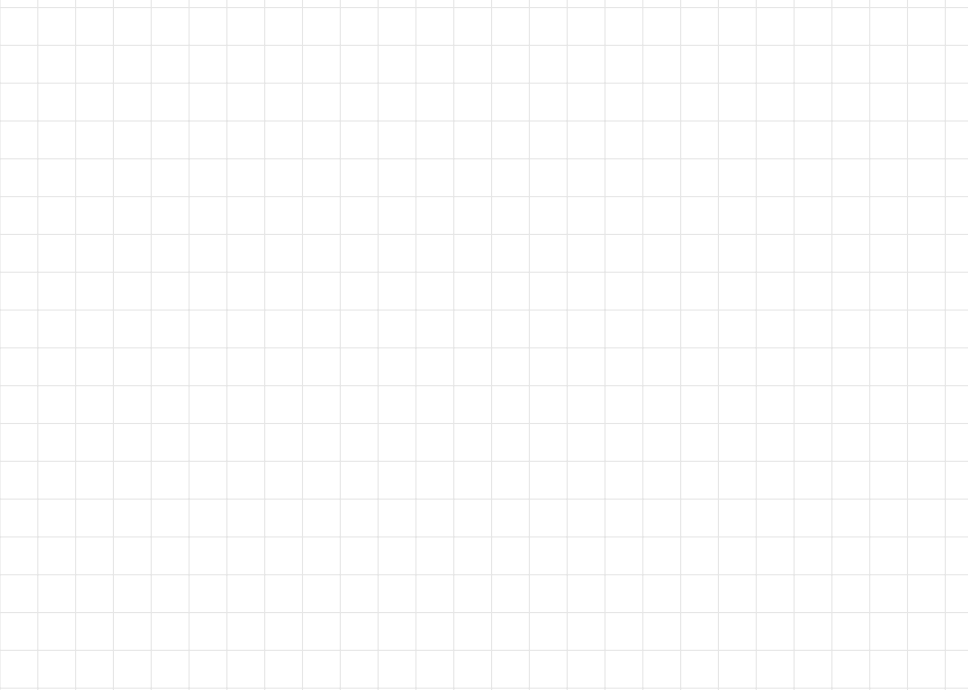
$$(2) P_2((x_i \in S_t) \cap \sim A) = P_2(x_i \in S | \sim A) \cdot \underbrace{P_2(\sim A)}_{1 - m/t}$$

$$P_2(x_i \in S_t | \sim A) = P_2(x_i \in S_{t-1}) = \frac{m}{t-1} \text{ by inductive Hp}$$

$$\Rightarrow P_2((x_i \in S_t) \cap \sim A) = \frac{m}{t-1} \cdot \left(1 - \frac{m}{t}\right)$$

By putting together all pieces, we conclude that:

$$\begin{aligned}
 P_2(x_i \in S_t) &= (1) + (2) \\
 &= \frac{m}{t-1} \left(1 - \frac{1}{n}\right) \cdot \frac{n}{t} + \frac{m}{t-1} \left(1 - \frac{m}{t}\right) \\
 &= \frac{m}{t-1} \left(\frac{m}{t} - \frac{1}{t} + 1 - \frac{m}{t}\right) \\
 &= \frac{m}{t-1} \left(1 - \frac{1}{t}\right) = \frac{m}{t} \quad \square
 \end{aligned}$$



Sampling-based applications: Frequent Items

Frequent Items problem

Given a stream $\Sigma = x_1, x_2, \dots, x_n$ of n items from a universe U and a frequency threshold $\varphi \in (0, 1)$, determine all distinct items that occur at least $\varphi \cdot n$ times in Σ (we call them **frequent items**).

Example : $n = 100$ $U = \{A, B, C, D, E\}$

* A : 30 occurrences

* B : 30 "

* C : 20 "

* D, E : 10 " each

$$q = 0.3 \Rightarrow$$

Frequent Items = $\{A, B\}$

OBSERVATIONS

1: Number of frequent items is $\leq 1/\varphi$

2: EXACT STRATEGY

- Use a Map (e.g. Hash Table) T which maintains the # occurrences of all distinct items which have been arrived so far
- At the end of the stream, return all items in T with $\geq \varphi \cdot n$ occurrences

Weaknesses of EXACT STRATEGY:

let $\bar{n}$ be the number of distinct items in Σ

If $\bar{n} \approx n$ but $1/y \ll n$ the strategy becomes inefficient in terms of both space and time requirements

Unfortunately, to determine exactly the frequent items and their counts spending time and space proportional to the number of distinct items seems necessary

Frequent Items with Reservoir sampling

Frequent items can be approximated through reservoir sampling:

- 1 Extract an m -sample S from Σ with reservoir sampling. Note that the same element may occur multiple times in the sample!
- 2 Return the subset $S' \subseteq S$ of distinct items in the sample

How well does S' approximate the set of frequent items?

- If $m \geq 1/\varphi$, for any given frequent item a , we expect to find at least one copy of a in S (hence in S').
- However, some frequent item may be missed, and S' may contain items with very low frequency.

Exercise

Let $\Sigma = x_1, x_2, \dots, x_n$ be a stream of n items and let S be an m -sample computed on Σ using reservoir sampling. For a given frequency threshold $\varphi \in (0, 1)$, consider a frequent item a , that is an item occurring at least φn times in Σ . Show that, if $m \geq 1/\varphi$, at least one occurrence of a is in S , in expectation.

Observation (relevant for the comparison with the Sticky sampling algorithm described later):
If $m \geq 1/\varphi$, at the end of the stream, the sampling rate becomes $\geq 1/(\varphi \cdot n)$

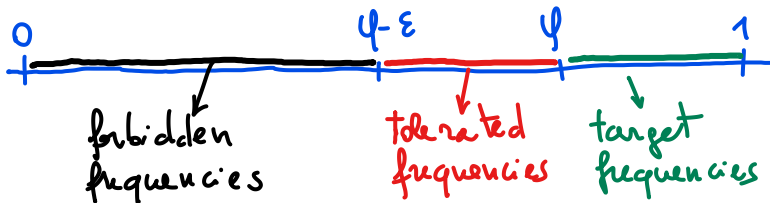
Approximate Frequent Items

ϵ -Approximate Frequent Items (ϵ -AFI) problem

Given the stream $\Sigma = x_1, x_2, \dots, x_n$ of n items, a frequency threshold $\varphi \in (0, 1)$, and an accuracy parameter $\epsilon \in (0, \varphi)$, return a set of distinct items that

- includes all items occurring at least $\varphi \cdot n$ times in Σ .
- contains no item occurring less than $(\varphi - \epsilon) \cdot n$ times in Σ .

Remark: with this formulation, we avoid false negatives but tolerate high-frequency false positives.



Example (same as before) $n = 100$

* A: 30 occurrences

* B: 30 "

* C: 20 "

* D, E: 10 " each

$$q = 0.3 \quad \varepsilon = 0.1$$

legal outputs:

$\{A, B\}$

$\{A, B, C\}$

Neither D and E can be included in the output

Sticky Sampling

Sticky Sampling provides a probabilistic solution to the ϵ -AFI problem.

MAIN INGREDIENTS:

- Confidence parameter $\delta \in (0, 1)$.
- Hash Table S , whose entries are pairs $(x, f_e(x))$ where
 - x (the key) is an item
 - $f_e(x)$ (the value) is an lower bound to the number of occurrences of x seen so far.
- Sampling rate chosen to ensure (with probability $\geq 1 - \delta$) the frequent items are in the sample.

Sticky Sampling: algorithm

Consider stream $\Sigma = x_1, x_2, \dots, x_n$, and assume n known!

Initialization:

- $S \leftarrow$ empty Hash Table;
- $r = \ln(1/(\delta\varphi))/\epsilon$; // *sampling rate* $= r/n$.

For each x_t **in** Σ **do**

if $(x_t, f_e(x_t)) \in S$ **then** $f_e(x_t) \leftarrow f_e(x_t) + 1$;
else add $(x_t, 1)$ to S with probability r/n ; /* *(start tracking x_t)* */

At the end: **return** all items x in S with $f_e(x) \geq (\varphi - \epsilon)n$

Sticky Sampling: analysis

Theorem

Sticky sampling solves the ϵ -AFI problem correctly with probability at least $1 - \delta$ and requires

- working memory of size $O(r) = O(\ln(1/(\delta\varphi))/\epsilon)$, in expectation;*
- 1 pass;*
- $O(1)$ update time in expectation.*
- $O(r) = O(\ln(1/(\delta\varphi))/\epsilon)$ query time in expectation.*

Remark: The space is independent of the stream length.

For fixed δ, ϵ the sampling rate will be $O(\frac{1}{\varphi n})$
 $\Rightarrow$ smaller sampling rate than the one
required by reservoir sampling

Proof.

* **WORKING MEMORY SIZE**: proportional to the number of entries in S (assuming each entry taking $O(1)$ space). Now each x_t contributes a new entry to S with probability $p_t \leq r/n$

$$X_t = \begin{cases} 1 & \text{if } x_t \text{ contributes a new entry to } S \\ 0 & \text{otherwise} \end{cases}$$

at the end of the stream $|S| = \sum_{t=1}^n X_t$

$$\Rightarrow E[|S|] = E\left[\sum_{t=1}^n X_t\right] = \sum_{t=1}^n E[X_t] = \sum_{t=1}^n p_t \leq n \frac{r}{n} = r$$

$\Rightarrow$ working memory size = $O(r)$

* 1 PASS, $O(1)$ UPDATE TIME (in expectation):

Straight forward

* QUERY TIME: $O(r)$ in expectation immediate
consequence of the fact that $E[|S'|] = O(r)$

* CORRECTNESS: we need to prove that with
probability $\geq 1 - \delta$ the returned output satisfies

(A) The output contains all true frequent items

(B) The output contains NO ITEM that occurs less than $(\psi - \epsilon)n$ times

Property (B) is IMMEDIATE by construction, since the output contains only items that in S have frequency value $\geq (\psi - \epsilon) \cdot n$, and frequency values in S are always lower bounds to the true frequencies $\Rightarrow$ We need to prove only Property (A)

PROOF that (A) holds with probability $\geq 1 - \delta$

let a be an arbitrary true frequent item

$$\begin{aligned}
& P_2(a \text{ is not returned in the output}) \\
& \leq P_2(\text{none of the first } \lceil \varepsilon n \rceil \text{ occurrences of } a \text{ are sampled}) \\
& = \left(1 - \frac{1}{n}\right)^{\lceil \varepsilon n \rceil} \leq \left(1 - \frac{1}{n}\right)^{\varepsilon n} \\
& = \left(1 - \frac{1}{n}\right)^{\varepsilon \cdot n \cdot \frac{1}{\varepsilon} \cdot \frac{1}{2}} = \left(\left(1 - \frac{1}{n/\varepsilon}\right)^{n/\varepsilon}\right)^{\varepsilon/2} \\
& \leq \left(\frac{1}{e}\right)^{\varepsilon/2} \quad \text{when we use the fact that} \\
& \quad \left(1 - \frac{1}{x}\right)^x \leq 1/e \quad \forall x \geq 1
\end{aligned}$$

let $a_1, a_2, \dots, a_k$ be all the frequent items
 we know $k \leq 1/\psi$

Now

$$\begin{aligned} & P_2(\exists \text{ some } a_i \text{ not returned in the output}) \\ & \leq \sum_{i=1}^k P_2(a_i \text{ is not returned in the output}) \\ & \leq k \left(\frac{1}{e}\right)^{\varepsilon k} \leq \frac{1}{y} \left(\frac{1}{e}\right)^{\varepsilon \cdot \ln(1/y\delta)/\varepsilon} \quad \swarrow \text{UNION BOUND} \\ & = \frac{1}{y} \left(\frac{1}{e}\right)^{\ln\left(\frac{1}{y\delta}\right)} = \frac{1}{y} \cdot \delta y = \delta \\ & \Rightarrow P_2(\text{all frequent items are returned}) \geq 1 - \delta \end{aligned}$$

Extension to n unknown ($\Rightarrow$ unbounded stream)

Main ingredients:

- * Dynamic sampling rate

- * Consider Σ divided into BATCHES

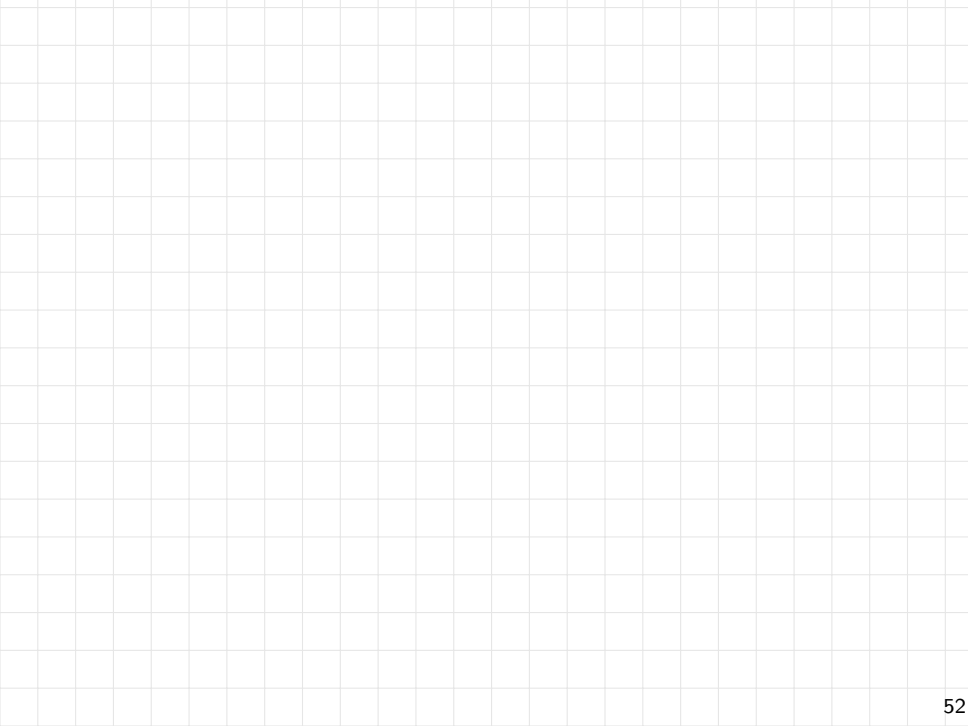
$$\Sigma = B_0 B_1 \dots B_i \dots$$

$$|B_i| = 2^i n \quad n = \ln(1/\delta) / \epsilon \text{ as before}$$

For B_i (i.e., for all items in B_i) the sampling rate will be $1/2^i$

- * Reallocation of the hash tables after each batch

We skip all details of the algorithm and
of the analysis



Exercise

Suppose that S_1 and S_2 are two m -samples extracted from two streams Σ_1 and Σ_2 of size n each. Given S_1 and S_2 , show how to construct an m -sample of the concatenation of the two streams $\Sigma_1 \cdot \Sigma_2$ (assume that all elements of the two streams are distinct).

Exercise

Let $S = \{s_1, s_2, \dots, s_m\}$ be an m -sample extracted from a stream Σ of n items labeled either *red* or *blue*. Let $R \in [1, n]$ be the number of red items in Σ and let X_S be the number of red items in S . Show that $(n/m) * X_S$ is an unbiased estimator of R , that is that $E[(n/m) * X_S] = R$.

Exercise

For a stream Σ of n items $x_1, x_2, \dots, x_n$, consider the following algorithm:

- Run in parallel the Boyer-Moore majority algorithm and the Sticky Sampling algorithm with $\varphi = 1/2$.
- Let **cand** be the candidate identified by the Boyer-Moore algorithm, and **S** the set of items returned by the Sticky Sampling algorithm.

If $\text{cand} \in S$ then return **cand otherwise return null.**

What can we say about the output of the algorithm? Distinguish among the case where Σ contains a majority element and the case where Σ does not contain a majority element.

References

- **[LRU14]** J. Leskovec, A. Rajaraman and J. Ullman. **Mining Massive Datasets**. Cambridge University Press, 2014. Chapter 4 (Sections 4.3-4.5) (pdf provided in Moodle)
- **[DF08]** C. Demetrescu and I. Finocchi. **Algorithms for Data Streams**. In *Handbook of Applied Algorithms*, Chapter 8, Section 3. Wiley-IEEE Press, 2008. (pdf provided in Moodle)
- **[CGHJ12]** G. Cormode, M.N. Garofalakis, P.J. Haas, C. Jermaine. **Synopses for Massive Data: Samples, Histograms, Wavelets, Sketches**. Foundations and Trends in Databases 4(1-3): 1-294, 2012. Chapter 5 (Sections 5.1-5.3) (pdf provided in Moodle)